

Pixel Level Transformation

Pixel Level Transformation

Preface

Example Results

Theory

convertScaleAbs

LUT Lookup Table

Code Overview

Import Modules and Initialize Hardware

Create LUT and Set Mode

Capture Image and Convert to Grayscale

Execute Pixel Transformation

Package Results and Display

Resource Release

Parameter Adjustment Notes

Preface

This tutorial applies to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. You can download it from the Kendryte official website at https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/

 **K230 MicroPython 镜像下载**

 CI/CD 更新时间: 2026-07-28 01:29:59

选择开发板

 CanMV K230 V1.0/V1.1

 CanMV K230 V3.0

 庐山派 K230

 庐山派 Lite K230D

 **Yahboom K230**

 01Studio K230

 01Studio K230 Emmc

 Hiwonder K230

 WonderMK K230

 GT6700 K230

 **勘智**
KENDRYTE

Yahboom K230

MicroPython Build for Yahboom K230

当前下载镜像:
[CanMV_K230_YAHBOOM_micropython_v1.8-28-g9b151af_nncase_v2.11.0.img.gz](#)

最新每日构建版本

Daily-0728

编译日期: 2026-07-28

MD5: 66f0622fc6b8d22e55f552a40458d2fa

 **立即下载**

极简 USB 烧录指南 (推荐)

 镜像仅可烧录到对应的开发板, 请务必选择与当前板型完全匹配的镜像, 不能混烧其他板型的镜像。否则可能导致硬件损坏!!!

1 准备工具

下载并打开 **K230BurningTool** 烧录工具, 并加载固件。

2 进入烧录模式 (根据硬件情况选择)

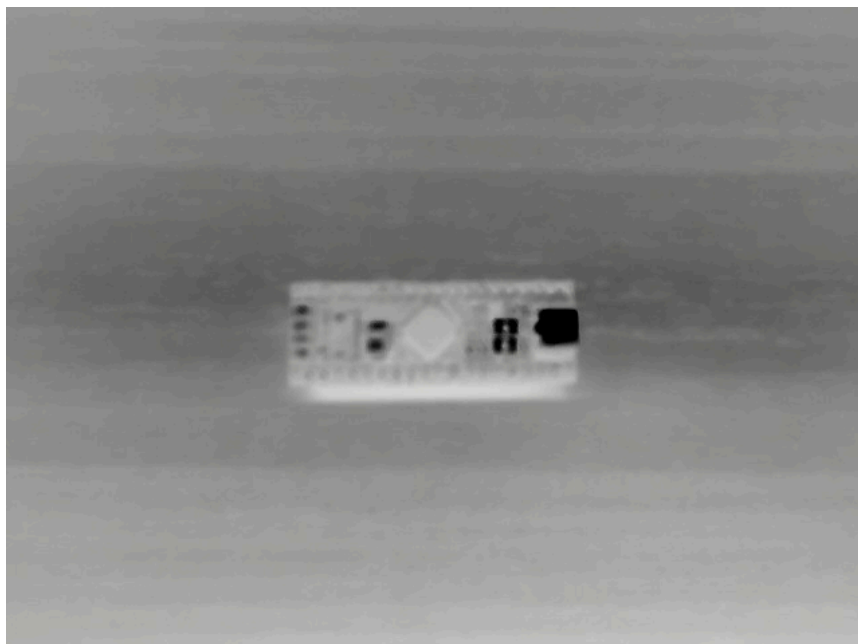
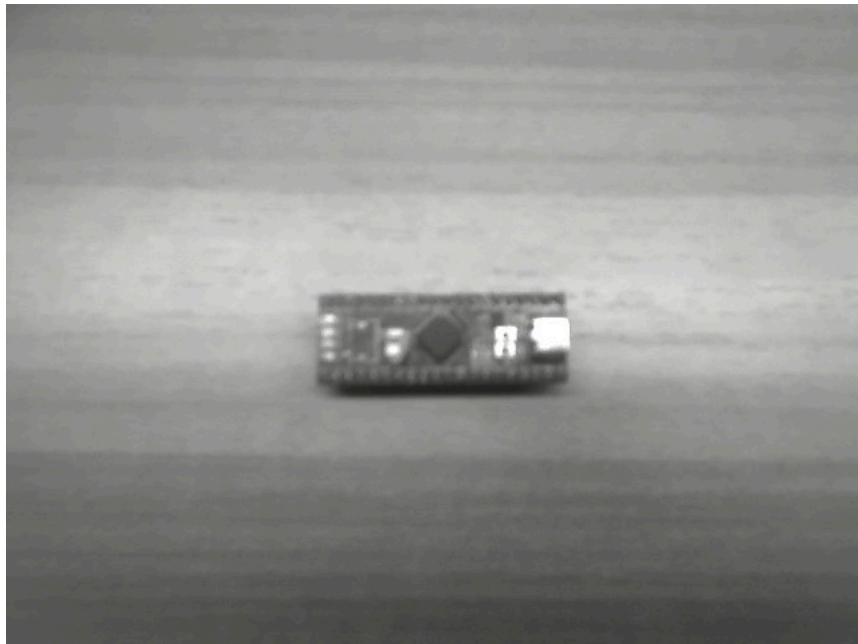
 有 BOOT 键的板子

 无 BOOT 键的板子

Example Results

Run the example code for this section: [17.camera_test_imgproc_new.py](#).

This section focuses on two pixel-level APIs: `convertScaleAbs()` uses fixed coefficients to adjust grayscale contrast and offset, and `LUT()` uses a 256-entry lookup table for inverse mapping. Both modes run on 640x480 grayscale live video.



Theory

`convertScaleAbs`

`convertScaleAbs()` performs scaling, offset, absolute value, and 8-bit saturation conversion for each pixel:

```
dst(x, y) = saturate_uint8(abs(alpha × src(x, y) + beta))
```

This example uses `alpha=1.5`, `beta=-30`. Mid-to-high grayscale differences are amplified, and darker regions first subtract 30. Since absolute value is taken afterward, it is not equivalent to ordinary "contrast multiplied by 1.5, brightness reduced by 30."

LUT Lookup Table

8-bit input has only 256 possible values, so you can prepare output for each input in advance:

```
lut[i] = 255 - i
```

This table implements grayscale inversion. By replacing with gamma, piecewise, or threshold mapping tables, other fixed transformations can be completed without running Python loops per pixel.

Code Overview

Import Modules and Initialize Hardware

```
import time, os, sys, gc
from media.sensor import *
from media.display import *
from media.media import *
import cv2
import ulab.numpy as np

W, H = 640, 480
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=W, height=H)
sensor.set_pixformat(Sensor.RGB888)
Display.init(Display.ST7701, width=640, height=480, to_ide=True)
sensor.run()
time.sleep(0.5)
```

Create LUT and Set Mode

```
lut = np.zeros((256, 1), dtype=np.uint8)
for i in range(256):
    lut[i, 0] = 255 - i

mode = 0
```

- mode=0: convertScaleAbs(), source default.
- mode=1: LUT() inverse.

LUT is generated once outside the main loop to avoid 256 Python assignments per frame.

Capture Image and Convert to Grayscale

```
img = sensor.snapshot()
img_np = img.to_numpy_ref()
gray = cv2.cvtColor(img_np, cv2.COLOR_BGR2GRAY)
```

Both transformations operate on single-channel 8-bit images for easy direct comparison of grayscale mapping results.

Execute Pixel Transformation

```
if mode == 0:
    out = cv2.convertScaleAbs(
        gray,
        alpha=1.5,
        beta=-30
    )
else:
    out = cv2.LUT(gray, lut)
```

convertScaleAbs() is determined by a fixed formula, while LUT() is determined by 256 values in the table, suitable for arbitrary discrete grayscale mappings.

Package Results and Display

```
img_out = image.Image(
    W, H, image.GRAYSCALE,
    alloc=image.ALLOC_REF,
    data=out
)
Display.show_image(img_out)
```

Resource Release

```
finally:
    sensor.stop()
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
```

Parameter Adjustment Notes

- **alpha:** Increasing expands grayscale differences but also more easily saturates bright areas; decreasing compresses contrast.
- **beta:** Controls linear offset, but this function also takes absolute value, so negative offset regions need to be understood in conjunction with the formula.
- **Inverse LUT:** 255-i applies to 8-bit images. When input depth changes, table length and type requirements need to be reconfirmed.
- **Gamma LUT:** Can be generated as $255 \times (i/255)^\gamma$; $\gamma < 1$ usually brightens dark areas, $\gamma > 1$ usually darkens midtones.
- **Piecewise mapping:** Can stretch the desired grayscale range to 0~255 while compressing other ranges, used to highlight specific brightness targets.
- **Difference from Lesson 14:** Lesson 14 compares equalization, normalization, and other global mappings. This lesson specifically explains convertScaleAbs and LUT's per-pixel mapping mechanism.